Jenkins-Style Parameterized Lambda Deployment Using AWS Native Services (PoC)

1. Overview

This document describes a **Proof of Concept (PoC)** to replace a **Jenkins "Build with Parameters" pipeline** with **AWS-native services**, while still supporting **dynamic user inputs** at runtime.

The solution enables on-demand deployment of **Python AWS Lambda code** using the following **user-provided parameters**:

- Lambda function name
- AWS region
- GitHub repository (private)
- Git branch

The entire flow is implemented **without Jenkins**, using managed AWS services only.

2. Problem Statement

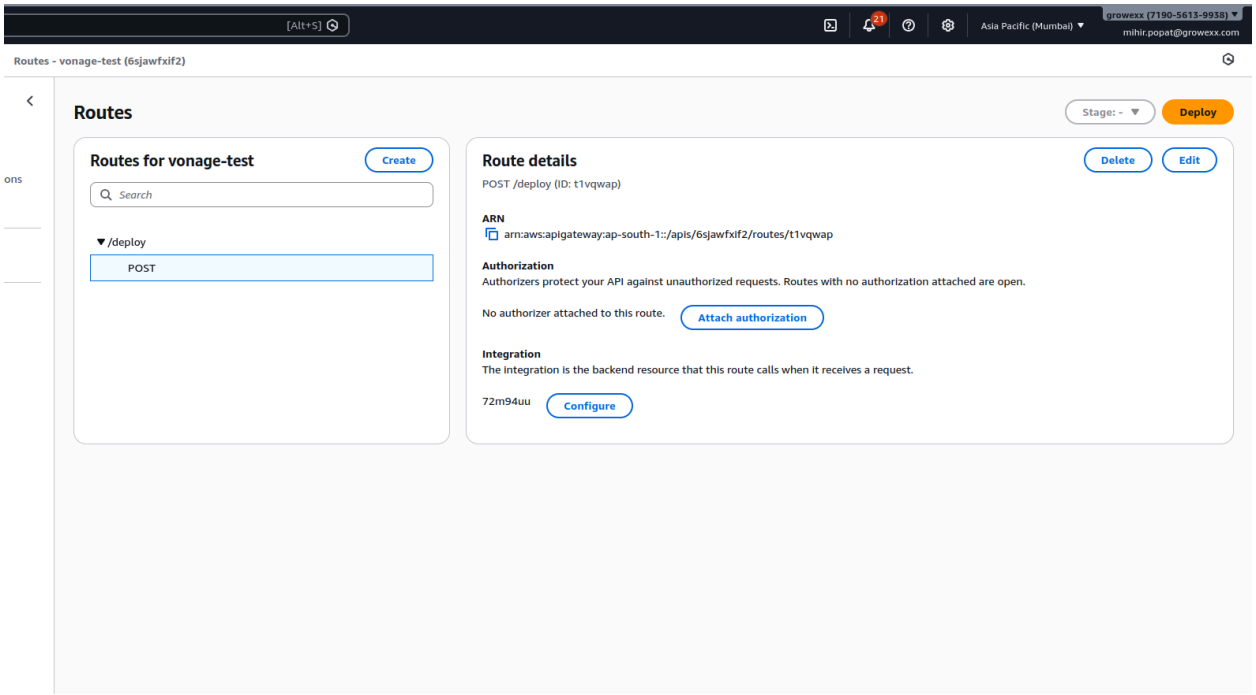
Jenkins provides a convenient UI for:

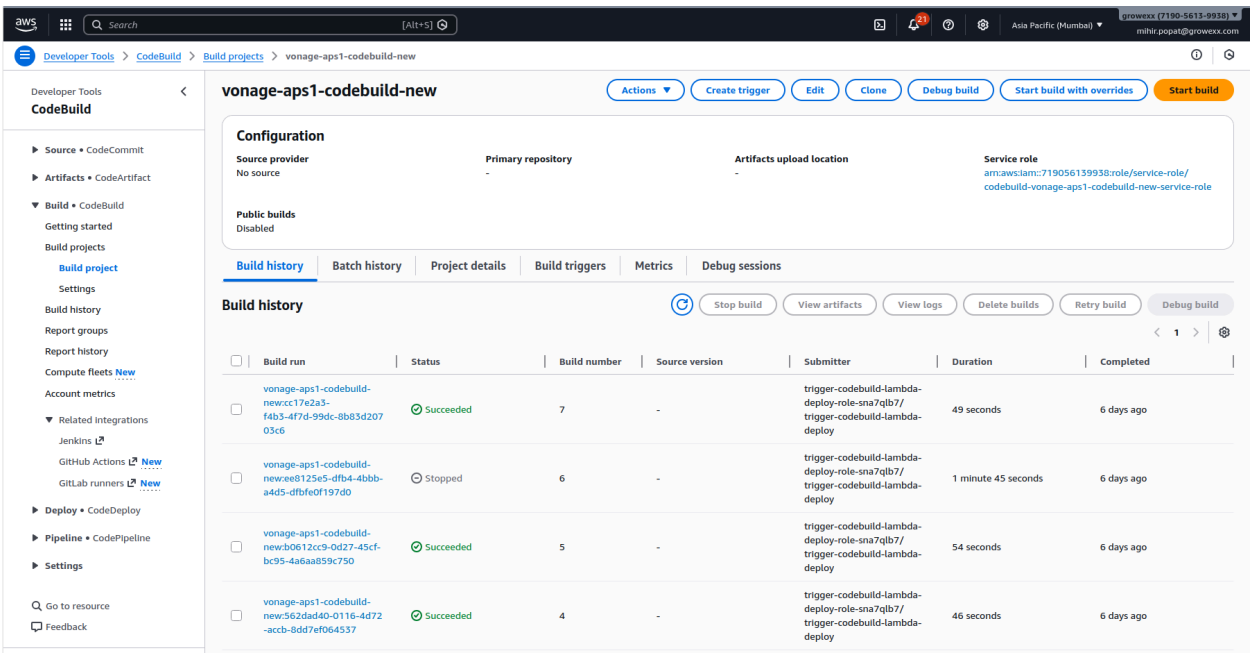
- Manual pipeline triggers
- Dropdowns / parameters
- Shell-based deployment logic

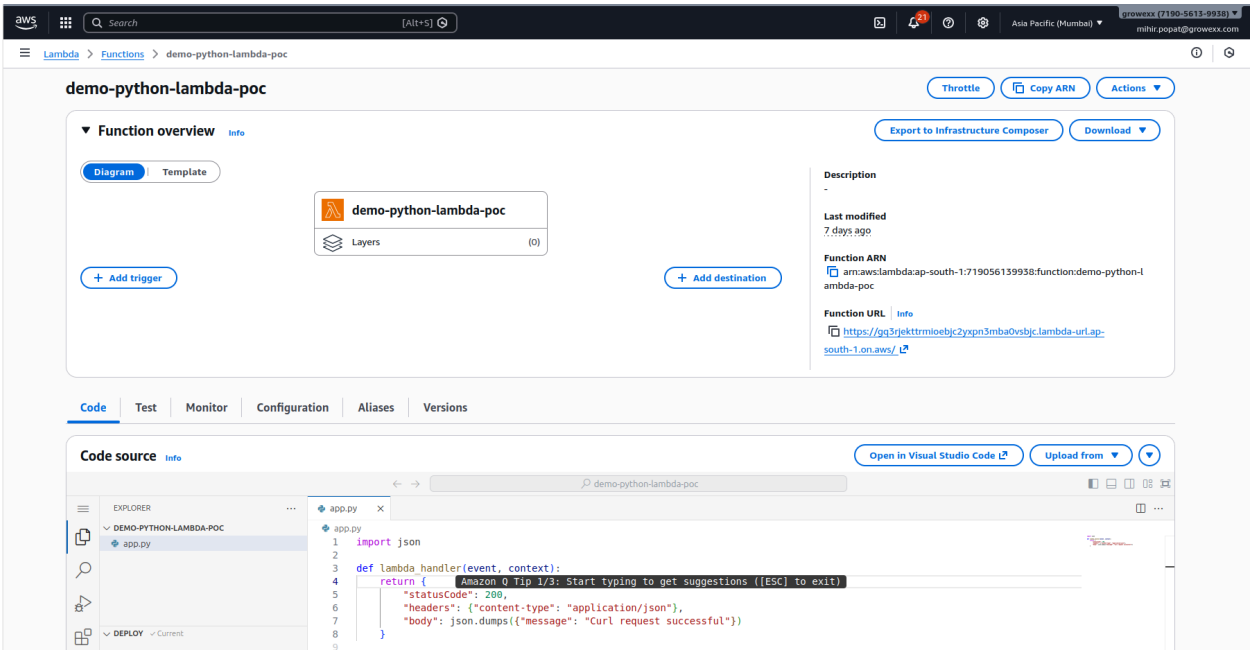
However, Jenkins introduces:

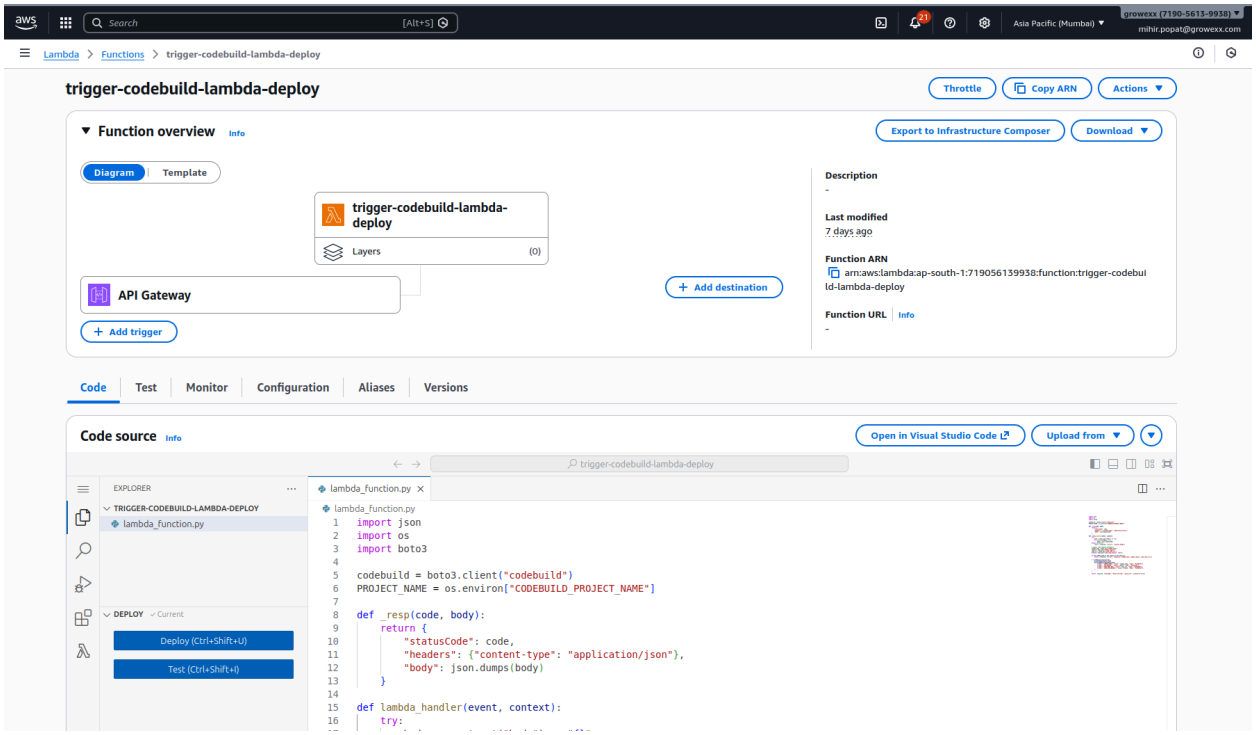
- Operational overhead (server, plugins, security patches)
- Credential management complexity
- Scaling and availability concerns

AWS native CI/CD services do not provide a **Jenkins-style UI**, but they support **parameterized execution via APIs**, which is more secure and automation-friendly.


[Alt+S]
Asia Pacific (Mumbai)
growexx (7190-5613-9938)
mihir.popat@growexx.com
Routes - vonage-test (6sjawfxif2)
Routes
Stage: -
Deploy
Routes for vonage-test
Create
Search
/deploy
POST
Route details
Delete
Edit
POST /deploy (ID: t1vqwap)
ARN
arn:aws:apigateway:ap-south-1::/apis/6sjawfxif2/routes/t1vqwap
Authorization
Authorizers protect your API against unauthorized requests. Routes with no authorization attached are open.
No authorizer attached to this route.
Attach authorization
Integration
The integration is the backend resource that this route calls when it receives a request.
72m94uu
Configure


aws
Search
[Alt+S]
Asia Pacific (Mumbai)
growexx (7190-5613-9938)
mihir.popat@growexx.com
Developer Tools > CodeBuild > Build projects > vonage-aps1-codebuild-new
Developer Tools
CodeBuild
Source • CodeCommit
Artifacts • CodeArtifact
Build • CodeBuild
Getting started
Build projects
Build project
Settings
Build history
Report groups
Report history
Compute fleets New
Account metrics
Related integrations
Jenkins
GitHub Actions New
GitLab runners New
Deploy • CodeDeploy
Pipeline • CodePipeline
Settings
Go to resource
Feedback
vonage-aps1-codebuild-new
Actions
Create trigger
Edit
Clone
Debug build
Start build with overrides
Start build
Configuration
Source provider
No source
Primary repository
-
Artifacts upload location
-
Service role
arn:aws:iam::719056139938:role/service-role/codebuild-vonage-aps1-codebuild-new-service-role
Public builds
Disabled
Build history
Batch history
Project details
Build triggers
Metrics
Debug sessions
Build history
Stop build
View artifacts
View logs
Delete builds
Retry build
Debug build
Build run
Status
Build number
Source version
Submitter
Duration
Completed
vonage-aps1-codebuild-new:cc17e2a3-f4b3-4f7d-99dc-8b83d20703c6
Succeeded
7
-
trigger-codebuild-lambda-deploy-role-sna7qlb7/trigger-codebuild-lambda-deploy
49 seconds
6 days ago
vonage-aps1-codebuild-new:ee8125e5-dfb4-4bbb-a4d5-dfbfe0f197d0
Stopped
6
-
trigger-codebuild-lambda-deploy-role-sna7qlb7/trigger-codebuild-lambda-deploy
1 minute 45 seconds
6 days ago
vonage-aps1-codebuild-new:b0612cc9-0d27-45cf-bc95-4a6aa859c750
Succeeded
5
-
trigger-codebuild-lambda-deploy-role-sna7qlb7/trigger-codebuild-lambda-deploy
54 seconds
6 days ago
vonage-aps1-codebuild-new:562dad40-0116-4d72-accb-8dd7ef064537
Succeeded
4
-
trigger-codebuild-lambda-deploy-role-sna7qlb7/trigger-codebuild-lambda-deploy
46 seconds
6 days ago


Lambda > Functions > demo-python-lambda-poc
demo-python-lambda-poc
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
demo-python-lambda-poc
Layers (0)
Add trigger
Add destination
Description
-
Last modified
7 days ago
Function ARN
arn:aws:lambda:ap-south-1:719056139938:function:demo-python-lambda-poc
Function URL
https://gq3rjekttrmioebjc2yxpn3mba0vsbjc.lambda-url.ap-south-1.on.aws/
Code
Test
Monitor
Configuration
Aliases
Versions
Code source
Open in Visual Studio Code
Upload from
EXPLORER
DEMO-PYTHON-LAMBDA-POC
app.py
import json
def lambda_handler(event, context):
return {
"statusCode": 200,
"headers": {"content-type": "application/json"},
"body": json.dumps({"message": "Curl request successful"})
}
DEPLOY


Lambda > Functions > trigger-codebuild-lambda-deploy
trigger-codebuild-lambda-deploy
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
trigger-codebuild-lambda-deploy
Layers (0)
API Gateway
Add trigger
Add destination
Description
-
Last modified
7 days ago
Function ARN
arn:aws:lambda:ap-south-1:719056139938:function:trigger-codebuild-lambda-deploy
Function URL
-
Code
Test
Monitor
Configuration
Aliases
Versions
Code source
Open in Visual Studio Code
Upload from
EXPLORER
TRIGGER-CODEBUILD-LAMBDA-DEPLOY
lambda_function.py
import json
import os
import boto3
codebuild = boto3.client("codebuild")
PROJECT_NAME = os.environ["CODEBUILD_PROJECT_NAME"]
def _resp(code, body):
return {
"statusCode": code,
"headers": {"content-type": "application/json"},
"body": json.dumps(body)
}
def lambda_handler(event, context):
try:
DEPLOY
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)

Buildspec.yml file

```
version: 0.2


env:
  secrets-manager:
    # SecretName:jsonKey
    GITHUB_TOKEN: "github/poc/token:token"


phases:
  install:
    runtime-versions:
      python: 3.11
    commands:
      - yum install -y git zip


  pre_build:
    commands:
      - echo "LAMBDA_NAME=$LAMBDA_NAME"
      - echo "AWS_REGION=$AWS_REGION"
      - echo "CODE_REPO_URL=$CODE_REPO_URL"
      - echo "CODE_REPO_BRANCH=$CODE_REPO_BRANCH"


      # Clone private GitHub repo using token
      - AUTH_REPO_URL=$(echo "$CODE_REPO_URL" | sed
"s#https://#https://$GITHUB_TOKEN@#")
```

```
    - rm -rf code || true
    - git clone --branch "$CODE_REPO_BRANCH" "$AUTH_REPO_URL" code


  build:
    commands:
    - cd code
    - mkdir -p package
    - if [ -f requirements.txt ]; then pip install -r requirements.txt -t package/; fi
    - cp -r src/* package/
    - cd package
    - zip -r ../lambda.zip .
    - cd ..


    - echo "DEBUG LAMBDA_NAME=[$LAMBDA_NAME] AWS_REGION=[$AWS_REGION]"
    - aws lambda update-function-code --function-name "$LAMBDA_NAME" --zip-file
"fileb://lambda.zip" --region "$AWS_REGION"
```

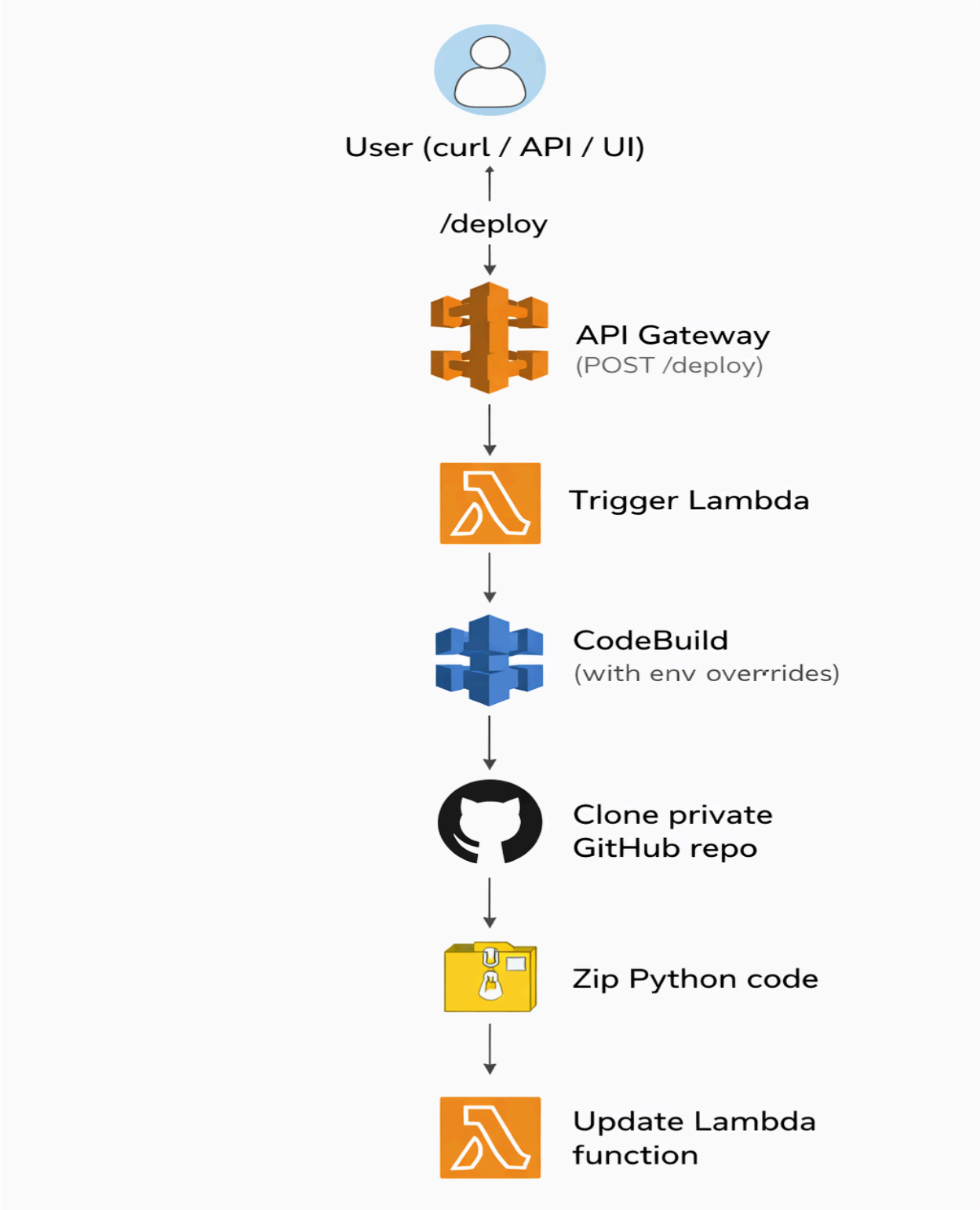
3. Solution Summary

This PoC implements a **Jenkins-equivalent parameterized pipeline** using:

- **Amazon API Gateway** – entry point for user input
- **AWS Lambda** – trigger function to process inputs
- **AWS CodeBuild** – build & deployment engine
- **AWS Secrets Manager** – secure GitHub token storage
- **GitHub** – private Lambda code repository

User inputs are passed at runtime, just like Jenkins parameters.

4. High-Level Architecture



Flow

User (curl / API / UI)

↓

API Gateway (POST /deploy)

↓

Trigger Lambda

↓

CodeBuild (with env overrides)

↓

Clone private GitHub repo

↓

Zip Python code

↓

Update Lambda function

5. Repository Design

Repo-1: Pipeline Repository

Purpose: Holds deployment logic only.

Contents: buildspec.yml

This repository **does not contain Lambda code**.

Repo-2: Lambda Code Repository (Private)

Purpose: Holds Python Lambda source code.

Structure:

src/

app.py
requirements.txt

Handler: app.handler

6. User Inputs (Jenkins Parameter Equivalent)

The following values are provided **per execution**:

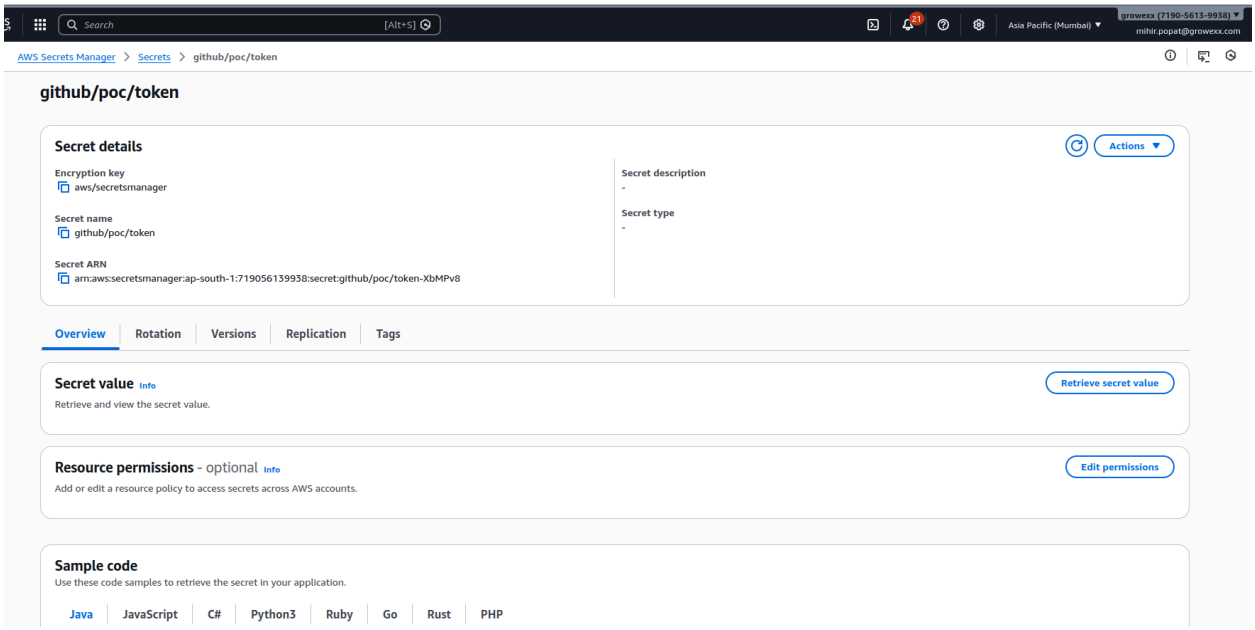
Parameter	Description
lambda_name	Target Lambda function name
lambda_region	AWS region
code_repo_url	Private GitHub repo URL
code_repo_branch	Git branch to deploy

These inputs are supplied as JSON to API Gateway.

7. Security Design

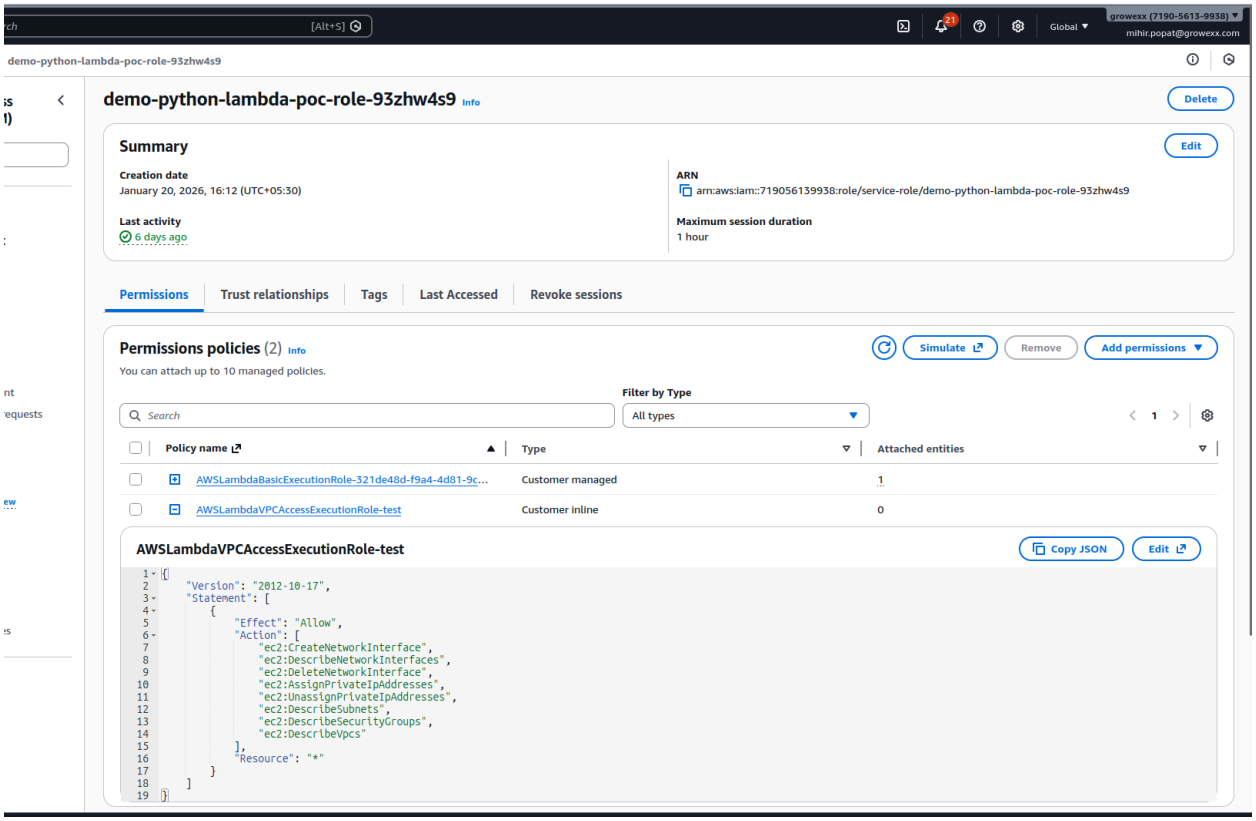
GitHub Authentication

- GitHub Personal Access Token (read-only)
- Stored securely in **AWS Secrets Manager**
- Injected into CodeBuild as a **secret environment variable**
- Never exposed in logs or plaintext



IAM Separation

- Trigger Lambda: codebuild:StartBuild
- CodeBuild:
    - lambda:UpdateFunctionCode
    - secretsmanager:GetSecretValue
    - CloudWatch Logs permissions



8. Build & Deployment Logic (CodeBuild)

Key Responsibilities

- Clone private GitHub repository
- Install dependencies
- Package Lambda code
- Deploy to the specified Lambda function and region

Deployment Command

```
aws lambda update-function-code \
  --function-name "$LAMBDA_NAME" \
  --zip-file fileb://lambda.zip \
  --region "$AWS_REGION"
```

All variables are supplied dynamically per run.

9. API Contract

Endpoint : POST /deploy

Request Body

```
{
  "lambda_name": "demo-python-lambda-poc",
  "lambda_region": "ap-south-1",
  "code_repo_url": "https://github.com/org/private-lambda-code-repo.git",
  "code_repo_branch": "main"
}
```

Response

```
{
  "message": "Build started",
  "build_id": "poc-lambda-deploy:uuid"
}
```

```
growlt328@growlt328:~$ curl -i -X POST "https://6sjawfxif2.execute-api.ap-south-1.amazonaws.com/dev/deploy" \
  -H "content-type: application/json" \
  -d '{
    "lambda_name": "demo-python-lambda-poc",
    "lambda_region": "ap-south-1",
    "code_repo_url": "https://github.com/mihir-growexxer/private-lambda-code-rep.git",
    "code_repo_branch": "demo"
  }'
HTTP/2 200
date: Tue, 27 Jan 2026 09:15:17 GMT
content-type: application/json
content-length: 106
apigw-requestid: X1lF0h9sBcwEPuw=

{"message": "Build started", "build_id": "vonage-aps1-codebuild-new:4504f4a4-81a5-4d50-9ac5-80af67738cb8"}growlt328@growlt328:~$
```

10. Execution Flow (Step-by-Step)

1. User sends deployment request to API Gateway
2. API Gateway invokes Trigger Lambda
3. Trigger Lambda validates inputs
4. Trigger Lambda starts CodeBuild with environment overrides
5. CodeBuild:
    - Fetches GitHub token from Secrets Manager
    - Clones private repo
    - Packages code
    - Updates Lambda
6. Logs are available in CloudWatch

11. Testing & Validation

Trigger Deployment

Verify

- CodeBuild build history
- CloudWatch logs
- Lambda “Last modified” timestamp
- Lambda invocation output

12. Limitations (PoC)

- No built-in UI dropdown (AWS is API-first)
- Manual JSON input required
- Single-step deployment (no approvals)

13. Future Enhancements

- Simple HTML UI with dropdowns (S3 + CloudFront)
- Step Functions for multi-stage workflows
- Manual approval steps
- GitHub Actions alternative
- IAM scope hardening
- Multi-account deployment support